CHAPTER 4

Abstract Data Structure

- 4.1 Linked list data structure

- REPRESENTED BY GROUP 1

CHAPTER 4 - 4.1. Linked-list data structure

Content Map

Introduction Linked-lists

The real-world C++ Demo

Cross-languages comparison

Exercises

- REPRESENTED BY GROUP 1

CHAPTER 4 - 4.1. Linked-list data structure

Introduce

- A linked list is a linear data structure which can store a collection of "nodes" connected together via links i.e. pointers.
- A linked list is a dynamic linear data structure whose memory size can be allocated or de-allocated at run time based on the operation insertion or deletion.

Compare with arrays

In case of arrays, the size is given at the time of creation and so arrays are of fixed lenghth where as Linked lists are dynamic in size and any number of nodes can be added in the linked lists dynamically. An array can accommodate similar types of data types where as linked lists can store various nodes of different data types.

CHAPTER 4 - 4.1. Linked-list data structure

Types of Linked-lists

4.1.1	**Simply Linked-list**	Definition • Representation • Operations
4.1.2	**Doubly Linked-list**	Definition • Representation • Operations
4.1.3	**Circular Linked-list**	Definition • Representation • Operations
4.1.4	**Sorted Linked-list**	Definition • Sort techniques • Comparison

- REPRESENTED BY GROUP 1

CHAPTER 4 – Abstract Data Structure

4.1.1 - Singly Linked- lists

- Definition
- Representation
- Operations

- REPRESENTED BY GROUP 1

CHAPTER 4 - 4.1. Linked-list data structure

Singly Linked-list

Singly linked lists contain two "buckets" in one node; one bucket holds the data and the other bucket holds the address of the next node of the list.

Structure

Node Structrue:

data: Value

next: Pointer to Node

List Structure:

NODE* pHead

NODE* pTail

Operations

- Traversal
- Insert at k position
- Delete at k position

CHAPTER 4 - 4.1. Linked-list data structure

Singly Linked-list

Algorthims descripstion

Traversal

1. START
2. While the list is not empty and did not reach the end of the list, print the data in each node
3. END

Search

1 START
2 If the list is not empty, iteratively check if the list contains the key
3 If the key element is not present in the list, unsuccessful search
4 END

- REPRESENTED BY GROUP 1

CHAPTER 4 - 4.1. Linked-list data structure

Singly Linked-list

Algorthims descripstion

Insert at head

1. START
2. Create a node to store the data
3. Check if the list is empty
4. If the list is empty, add the data to the node and assign the head pointer to it.
5. If the list is not empty, add the data to a node and link to the current head. Assign the head to the newly added node.
6. END

Delete at head

1. START
2. Assign the head pointer to the next node in the list
3. END

- REPRESENTED BY GROUP 1

CHAPTER 4 - 4.1. Linked-list data structure

Singly Linked-list

Algorthims descripstion

Insert at tail

1. START
2. Create a new node and assign the data
3. Find the last node
4. Point the last node to new node
5. END

Delete at tail

1. START
2. Iterate until you find the second last element in the list.
3. Assign NULL to the second last element in the list.
4. END

CHAPTER 4 - 4.1. Linked-list data structure

Singly Linked-list

Algorthims descripstion

Insert at k position

1. START
2. Create a new node and assign data to it
3. Iterate until the node at position is found
4. Point first to new first node
5. END

Delete by value

1. START
2. Iterate until find the current node at position in the list.
3. Assign the adjacent node of current node in the list to its previous node.
4. END

CHAPTER 4 - 4.1. Linked-list data structure

Singly Linked-list Complexity

Operation	Best Case	Average Case	Worst Case	Space
Traversal	*O(1)*	*O(n)*	*O(n)*	*O(1)*
Find / Search	*O(1)*	*O(n)*	*O(n)*	*O(1)*
Add Head	*O(1)*	*O(1)*	*O(n)*	*O(1)*
Add Tail	*O(1)*	*O(1)/O(n)*	*O(1)/O(n)*	*O(1)*
Add at Position *k*	*O(1)*	*O(k)*	*O(n)*	*O(1)*
Delete Head	*O(1)*	*O(1)*	*O(1)*	*O(1)*
Delete Tail	*O(1)*	*O(n)*	*O(n)*	*O(1)*
Delete at Position *k*	*O(1)*	*O(k)*	*O(n)*	*O(1)*

- REPRESENTED BY GROUP 1

CHAPTER 4 – Abstract Data Structure

4.1.2 - Doubly Linked- lists

- Definition
- Representation
- Operations

- REPRESENTED BY GROUP 1

CHAPTER 4 - 4.1. Linked-list data structure

Doubly Linked-list

Doubly Linked List is a variation of Linked list in which navigation is possible in both ways, forward as well as backward easily as compared to Single Linked List.

Structure

Node Structrue:

data: Value

next: Pointer to Node

prev: Pointer to Node

List Structure:

NODE* pHead

NODE* pTail

Operations

- Insert at head
- Delete at head
- Insert at end
- Delete at end

CHAPTER 4 - 4.1. Linked-list data structure

Doubly Linked-list

Algorthims descripstion

Insert at head

1. START
2. Create a new node with three variables: prev, data, next.
3. Store the new data in the data variable
4. If the list is empty, make the new node as head.
5. Otherwise, link the address of the existing first node to the next variable of the new node, and assign null to the prev variable.
6. Point the head to the new node.
7. END

Delete at head

1. START
2. Check the status of the doubly linked list
3. If the list is empty, deletion is not possible
4. If the list is not empty, the head pointer is shifted to the next node.
5. END

- REPRESENTED BY GROUP 1

CHAPTER 4 - 4.1. Linked-list data structure

Doubly Linked-list

Algorthims descripstion

Insert at tail

1. START
2. If the list is empty, add the node to the list and point the head to it.
3. If the list is not empty, find the last node of the list.
4. Create a link between the last node in the list and the new node.
5. The new node will point to NULL as it is the new last node.
6. END

Delete at tail

1. START
2. Assign the head pointer to the next node in the list
3. END

- REPRESENTED BY GROUP 1

CHAPTER 4 - 4.1. Linked-list data structure

Doubly Linked-list Complexity

Operation	Best Case	Average Case	Worst Case	Space
Traversal	*O(1)*	*O(n)*	*O(n)*	*O(1)*
Find / Search	*O(1)*	*O(n)*	*O(n)*	*O(1)*
Add Head	*O(1)*	*O(1)*	*O(1)*	*O(1)*
Add Tail	*O(1)*	*O(1)*	*O(1)*	*O(1)*
Add at Position *k*	*O(1)*	*O(min(k,n−k))*	*O(n)*	*O(1)*
Delete Head	*O(1)*	*O(1)*	*O(1)*	*O(1)*
Delete Tail	*O(1)*	*O(1)*	*O(1)*	*O(1)*
Delete at Position *k*	*O(1)*	*O(min(k,n−k))*	*O(n)*	*O(1)*

- REPRESENTED BY GROUP 1

CHAPTER 4 – Abstract Data Structure

4.1.3 - Circular Linked- lists

- Definition
- Representation
- Operations

- REPRESENTED BY GROUP 1

CHAPTER 4 - 4.1. Linked-list data structure

Circular Linked-list

Circular Linked List is a variation of Linked list in which the first element points to the last element and the last element points to the first element.
Both Singly Linked List and Doubly Linked List can be made into a circular linked list.

Structure

Node Structrue:

data: Value

next: Pointer to Node

prev: Pointer to Node

List Structure:

NODE* pHead

NODE* pTail

Operations

- Insert
- Delete
- Traversal

Circular Linked-list

Algorthims descripstion

Insert Operation

1. START
2. Check if the list is empty
3. If the list is empty, add the node and point the head to this node
4. If the list is not empty, link the existing head as the next node to the new node.
5. Make the new node as the new head.
6. END

Delete Operation

1. START
2. If the list is empty, then the program is returned.
3.END

- REPRESENTED BY GROUP 1

Circular Linked-list

Algorthims descripstion

Insert Operation

1. START
2. Check if the list is empty
3. If the list is empty, add the node and point the head to this node
4. If the list is not empty, link the existing head as the next node to the new node.
5. Make the new node as the new head.
6. END

Delete Operation

1. START
2. If the list is not empty, we traverse the list using a current pointer that is set to the head pointer and create another pointer previous that points to the last node.
3. END

Circular Linked-list

Algorthims descripstion

Displaying the List

1. START
2. Walk through all the nodes of the list and print them
3. END

CHAPTER 4 - 4.1. Linked-list data structure

Circular Linked-list Complexity

Function / Operation	Best Case	Average Case	Worst Case	Space Complexity
Traversal	***O(1)***	***O(1)***	***O(n)***	***O(1)***
AddHead	***O(1)***	***O(1)***	***O(1)***	***O(1)***
AddTail	***O(1)***	***O(1)***	***O(1)***	***O(1)***
Add_at_k_position	***O(1)***	***O(n)***	***O(n)***	***O(1)***
DeleteAtHead	***O(1)***	***O(1)***	***O(1)***	***O(1)***
DeleteAtTail	***O(1)***	***O(1)***	***O(1)***	***O(1)***
DeleteByValue	***O(1)***	***O(n)***	***O(n)***	***O(1)***

- REPRESENTED BY GROUP 1

CHAPTER 4 – Abstract Data Structure

4.1.4 - Sorted Linked- lists

- Sort Techniques
- Compare with Unsorted Lists

- REPRESENTED BY GROUP 1

CHAPTER 4 - 4.1. Linked-list data structure

Sort Linked-list

A Sorted Linked List is a type of linked list in which the nodes are arranged and always maintained in a specific order, such as ascending or descending based on their data values.

Sorted Techniques: Merge Sort

- Rewritten Point:

Merging two linked lists only rearranges existing node pointers, reducing auxiliary space complexity to *O(1)* instead of requiring *O(n)* extra memory as arrays do.

- Rewritten Point:

Merge Sort operates by traversing elements sequentially, making it perfectly compatible with Linked Lists' memory access model without requiring Random Access.

CHAPTER 4 - 4.1. Linked-list data structure

Sort Linked-list

Compare with Unsorted Linked-Lists

Criteria	Unsorted Linked List	Sorted Linked List
Insertion	***O(1)*** (Inserts directly at the head or tail)	***O(n)*** (Must traverse to find the correct sorted position)
Successful Search	***O(n)*** (Traverses from start to end)	***O(n)*** (Faster in practice due to early exit)
Unsuccessful Search	***O(n)*** (Must traverse the entire list)	***O(n)*** (Stops as soon as it meets a node larger than the target)
Find Min / Max	***O(n)*** (Requires traversing the whole list)	***O(1)*** (Directly access the head or tail node)
Deletion	***O(n)*** (Search *O(n)* + Delete *O(1)*)	***O(n)*** (Search *O(n)* + Delete *O(1)*)
Maintenance Cost	Low (No structural ordering rules to maintain)	High (Every insertion/update must preserve sorted order)

- REPRESENTED BY GROUP 1

CHAPTER 4 – 4.1. LINKED-LISTS DATA STRUCTURE

Advantages, Disadvantages of Linked- lists

- REPRESENTED BY GROUP 1

CHAPTER 4 - 4.1. Linked-list data structure

Advantges and Disadvantages of Linked-lists

Linked lists are a powerful and flexible data structure, but they have certain disadvantages that need to be taken into consideration when deciding whether to use them or not.

Advantages

- Efficient Insertion and Deletion *O(1)*
- Versatile Abstract Data Structure Base
- Flexible Memory Utilization
- No Array Reallocation Overhead

Disadvantages

- Slow Access Time
- Pointers or References
- Higher overhead
- Cache Inefficiency

CHAPTER 4 – Abstract Data Structure

Applications uses Linked-lists in real-world

- REPRESENTED BY GROUP 1

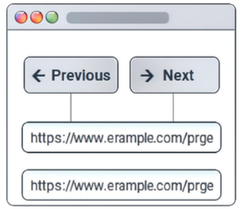
CHAPTER 4 - 4.1. Linked-list data structure

Application of Linked-lists



Music Player

The list of songs in the music player are linked to the previous and next songs.



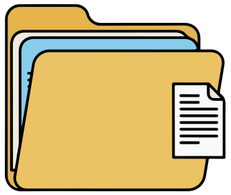
Web navigation features

In a web browser, previous and next web page URLs can be linked through the previous and next buttons



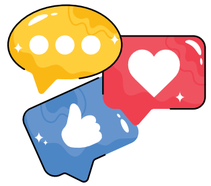
Undo/re-do

Each change is a node in a linked list (or doubly linked list), pointing to the next and previous states.



File System Management

Directory structures and find allocation tables can use linked-lists.



Social Media Feeds

Linked structures help in representing “next post” relationships dynamically

- REPRESENTED BY GROUP 1

CHAPTER 4 – Abstract Data Structure

The real - world C++ Demo

- Source code
- Test condition
- Report

- REPRESENTED BY GROUP 1

CHAPTER 4 - 4.1. Linked-list data structure

The real-world C++ demo

Problems

Managing web navigation features (visit, back, forward) using doubly linked-list

Test Condition

- Hardware setup:
  + CPU: Intel Core i5-11300H
  + RAM: 8 GB DDR4
- Software & Environment:
  + OS: Windows 11
  + Complier: C++17 standard
- Dataset:
  + *N = 1,000,000* randomly generated

URL operations

Report

Operation	Input Size (n)	Operation Counts	Wall-clock Time (Best of 5)	Peak Memory
Visit Node	1,000,000	1,000,000 ops	~630 ms	~113.46 MB
Back Traversal	200,000	200,000 steps	~5 ms	Neglibible
Forward Traversal	100,000	100,000 steps	~1 ms	Neglibible

- REPRESENTED BY GROUP 1

CHAPTER 4 – Abstract Data Structure

Cross - languages Comparison

- Benchmarks

- REPRESENTED BY GROUP 1

CHAPTER 4 - 4.1. Linked-list data structure

Cross-languages comparison

Problems

Empirical Performance Benchmark for Insertion and Search Operations on a Singly Linked List

Criterion	C++	Python	Java	JavaScript	Go
Built-in structure used	std::forward_list / Node*	Class w/ next ref	LinkedList<T> / Custom	Class w/ next ref	Struct w/ *Node
Insert Time (ms)	98.5	1180.0	165.0	215.0	130.0
Search Time (ms)	2.1	42.5	7.4	12.8	3.6
Total Execution Time (ms)	**100.6**	**1222.5**	**172.4**	**227.8**	**133.6**
Memory Footprint (MB)	**12.0**	**80.0**	**32.0**	**40.0**	**20.0**
Memory management	Manual (new/delete)	Garbage Collected	Garbage Collected	Garbage Collected	Garbage Collected
Type safety	Static & Strong	Dynamic & Strong	Static & Strong	Dynamic & Loose	Static & Strong
Readability of the code	Moderate	High	Moderate	High	High
Where you would use it	Embedded / High-Perf Core	Prototyping / Algorithms	Enterprise Backend Systems	Web Applications / Node.js	Cloud Services / Microservices

- REPRESENTED BY GROUP 1

CHAPTER 4 – Abstract Data Structure

Exercise

- REPRESENTED BY GROUP 1